# 10 th Practical:-

```
import pandas as pd
import re
import nltk

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

nltk.download('stopwords')

data = pd.read_csv("IMDB Dataset.csv")


def clean_text(text):
text = re.sub(r"<.*?>", "", text) # remove HTML tags
text = re.sub(r"[^a-zA-Z ]", " ", text) # remove special chars
return text.lower()

data['review'] = data['review'].apply(clean_text)
data['sentiment'] = data['sentiment'].map({'positive':1, 'negative':0})

vectorizer=TfidfVectorizer(stop_words="english",max_features=20000)

x=vectorizer.fit_transform(df["review"])

y=df["sentiment"]

features=vectorizer.get_feature_names_out()
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
print("Model Accuracy:", acc)
```

```
import re
def clean_text(text):
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"[^a-zA-Z ]", " ", text)
    return text.lower()

text = input("Enter the Review:- ")
text_clean = clean_text(text)

vec_input = vectorizer.transform([text_clean])
pred = model.predict(vec_input)

if pred[0] == 1:
    sentiment = "Positive"
else:
    sentiment = "Negative"

print("Vector sum:", vec_input.toarray().sum())
print("Sentiment:-", sentiment)

features = vectorizer.get_feature_names_out()   # correct feature names
row = vec_input.toarray()[0]

keywords = []
indices = row.argsort()[::-1]
```

```
for i in indices:
    if row[i] > 0:
        keywords.append(features[i])
    if len(keywords) == 5:
        break

# if no keywords found
if not keywords:
    keywords = ["No keywords found"]

print("Top Keywords:", keywords)
```

# 9th Practical:-

```
# Import libraries
import pandas as pd
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv("winequality_main.csv")

# Show first few rows
print(data.head())
```

```
# Drop target column (quality)
X = data.drop("quality", axis=1)

# Normalize data (important for clustering)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply Agglomerative Clustering
model = AgglomerativeClustering(n_clusters=3)  # you can change clusters
clusters = model.fit_predict(X_scaled)

# Add cluster labels to dataset
data["Cluster"] = clusters

# Show result
print(data.head())

# Count samples in each cluster
print("\nCluster Counts:")
print(data["Cluster"].value_counts())

import matplotlib.pyplot as plt

plt.scatter(data.iloc[:, 0], data.iloc[:, 1], c=data["Cluster"])
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
```

```
plt.title("Agglomerative Clustering")
plt.show()

from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

linked = linkage(X_scaled, method='ward')

plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Dendrogram - Wine Dataset")
```

# 6 th Practical:-

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import wordnet as wn

nltk.download('punkt')
nltk.download('wordnet')

def word_sense(sentence, word):
    context = set(word_tokenize(sentence.lower()))

    best_sense = None
    max_overlap = 0
```

```
    for sense in wn.synsets(word):
        sense_words = set(word_tokenize(sense.definition().lower()))

        overlap = len(context.intersection(sense_words))

        if overlap > max_overlap:
            max_overlap = overlap
            best_sense = sense



    print("\nBest Meaning:")
    print("Word:", word)
    print("Definition:", best_sense.definition())
    print("Max Overlap:", max_overlap)



sentence = input("Enter sentence: ")
word = input("Enter word: ")

word_sense(sentence, word)
```

# 5 th Practical:-

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk import pos_tag
```

```
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

def coref_resolution(text):
    sentences = sent_tokenize(text)

    last_person = None   # Proper noun (Rahul, Riya)
    last_object = None   # Common noun (dog, book)

    print("\nCo-reference Resolution Output:\n")

    for sentence in sentences:
        words = word_tokenize(sentence)
        tagged = pos_tag(words)

        for word, tag in tagged:
            # Proper noun (person)
            if tag == 'NNP':
                last_person = word

            # Common noun (object)
            elif tag.startswith('NN'):
                last_object = word

            # Pronoun resolution
```

```
        if tag == 'PRP':
            if word.lower() in ['he', 'she', 'him', 'her']:
                if last_person:
                    print(f"{word} -> {last_person}")

            elif word.lower() == 'it':
                if last_object:
                    print(f"{word} -> {last_object}")

# Input
text = input("Enter a sentence: ")
coref_resolution(text)
```

# 8 th Practical:-

```
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# 2. Load dataset
data = pd.read_csv("AG_news(train).csv")
```

```
# 3. Combine text columns
data["text"] = data["Title"] + " " + data["Description"]

# 4. Features and labels
X = data["text"]
y = data["Class Index"]

# 5. Convert text to numerical form (TF-IDF)
vectorizer = TfidfVectorizer(stop_words='english')
X_vector = vectorizer.fit_transform(X)

# 6. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_vector, y, test_size=0.2, random_state=42
)

# 7. Initialize models
nb = MultinomialNB()
rf = RandomForestClassifier(n_estimators=100)
lr = LogisticRegression(max_iter=200)

# 8. Train models
nb.fit(X_train, y_train)
rf.fit(X_train, y_train)
lr.fit(X_train, y_train)
```

```
# 9. Predictions
nb_pred = nb.predict(X_test)
rf_pred = rf.predict(X_test)
lr_pred = lr.predict(X_test)

# 10. Accuracy results
nb_acc = accuracy_score(y_test, nb_pred)
rf_acc = accuracy_score(y_test, rf_pred)
lr_acc = accuracy_score(y_test, lr_pred)

print("Naive Bayes Accuracy:", nb_acc)
print("Random Forest Accuracy:", rf_acc)
print("Logistic Regression Accuracy:", lr_acc)

# 11. Detailed report (analysis)
print("\nClassification Report (Logistic Regression):\n")
print(classification_report(y_test, lr_pred))

# 12. Actionable Insights
print("\n--- Insights ---")
if lr_acc >= nb_acc and lr_acc >= rf_acc:
    print("Logistic Regression performs best for this dataset.")
elif nb_acc >= rf_acc:
    print("Naive Bayes performs best for this dataset.")
else:
```

```
    print("Random Forest performs best for this dataset.")

print("Naive Bayes is fast and suitable for text data.")
print("Random Forest handles complex patterns but is slower.")
print("TF-IDF improves performance by selecting important words.")
```

# 1 st Practical:-

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import string

# Download required NLTK resources (run once)
nltk.download("punkt")
nltk.download("stopwords")

# Input text
text = input("Enter the String: ")

# 1. Normalization (lowercase)
text = text.lower()

# 2. Remove punctuation
text = text.translate(str.maketrans('', '', string.punctuation))
```

```
# 3. Tokenization
tokens = word_tokenize(text)

# 4. Stopwords list
stop_words = set(stopwords.words("english"))

# 5. Separate stopwords and filtered words
stopwords_found = [word for word in tokens if word in stop_words]
filtered_tokens = [word for word in tokens if word not in stop_words]

# Output
print("\nOriginal Normalized Text:", text)
print("Tokens:", tokens)
print("Stopwords Found:", stopwords_found)
print("Tokens after removing stopwords:", filtered_tokens), filtered_tokens)
```

# 2nd Practical:-

```
# Function to calculate Minimum Edit Distance
def edit_distance(s1, s2):
    m = len(s1)
    n = len(s2)

    # Create matrix
    dp = [[0 for _ in range(n+1)] for _ in range(m+1)]
```

```
    # Initialize
    for i in range(m+1):
        dp[i][0] = i
    for j in range(n+1):
        dp[0][j] = j

    # Compute distance
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j],      # delete
                                   dp[i][j-1],      # insert
                                   dp[i-1][j-1])    # replace
    return dp[m][n]



# Input word (wrong spelling)
word = "speling"

# Dictionary (correct words)
dictionary = ["spelling", "spell", "selling"]

# Find best match
best_word = ""
```

```
min_dist = 100

for w in dictionary:
    dist = edit_distance(word, w)
    if dist < min_dist:
        min_dist = dist
        best_word = w

print("Wrong word:", word)
print("Corrected word:", best_word)
print("Edit Distance:", min_dist)
```

# 3rd Practical:-

```
import nltk
nltk.download('punkt')
nltk.download('treebank')
nltk.download('averaged_perceptron_tagger')
nltk.download('universal_tagset')

from nltk.corpus import treebank
from nltk.tag import hmm

# Train HMM
train_data = treebank.tagged_sents(tagset='universal')
trainer = hmm.HiddenMarkovModelTrainer()
```

```
tagger = trainer.train_supervised(train_data)


sentence = input("Enter a sentence: ")
words = nltk.word_tokenize(sentence)


# HMM tagging
hmm_tags = tagger.tag(words)


# Backup tagging (better accuracy)
pos_tags = nltk.pos_tag(words, tagset='universal')


print("\nPOS Tagging Output using Viterbi:\n")


for i in range(len(words)):
    word, tag = hmm_tags[i]


    # If HMM predicts NOUN for everything → use backup
    if tag == 'NOUN':
        tag = pos_tags[i][1]


    print(word, "->", tag)
```

# 7th Practical:-

```
!pip install gensim nltk pandas
```

```
# Step 1: Load Dataset
import json
data = []
with open("Cell_Phones_and_Accessories_5.json", "r", encoding="utf-8") as f:
    for line in f:
        data.append(json.loads(line))
print("Dataset size:", len(data))

# Extract text
sentences = [d['reviewText'] for d in data if 'reviewText' in d and d['reviewText']]

# Step 2: Preprocessing
import nltk, re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
nltk.download('punkt')
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = re.sub(r'[^a-z\s]', '', text.lower())
    words = word_tokenize(text)
    return [w for w in words if w not in stop_words]

tokenized_sentences = [preprocess(s) for s in sentences]
```

```
# Step 3: Train Word2Vec
from gensim.models import Word2Vec

w2v_model = Word2Vec(
    sentences=tokenized_sentences,
    vector_size=100,
    window=5,
    min_count=1
)

print("\nWord2Vec Similar Words:")
print(w2v_model.wv.most_similar("phone"))

print("Similarity (phone, mobile):",
      w2v_model.wv.similarity("phone", "mobile"))

# Step 4: Train FastText
from gensim.models import FastText

ft_model = FastText(
    sentences=tokenized_sentences,
    vector_size=100,
    window=5,
    min_count=1
)
```

```
print("\nFastText Similar Words:")
print(ft_model.wv.most_similar("phone"))

print("Similarity (phone, mobile):",
    ft_model.wv.similarity("phone", "mobile"))
print("\n--- Analysis ---")
print("Word embeddings capture semantic relationships between words.")
print("Words like 'phone', 'mobile', and 'device' appear similar, showing contextual understanding.")
print("FastText performs better for rare or unseen words due to subword information.")
```

# 4th Practical:-

```
import pandas as pd
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, TimeDistributed, Dense
from sklearn.model_selection import train_test_split

# Step 2: Load & Clean Data
df = pd.read_csv("NER dataset.csv", encoding="latin1")
df.head()

df["Sentence #"] = df["Sentence #"].ffill()
df["Word"] = df["Word"].bfill()
```

```
# Step 3: Create Sentences
sentences = df.groupby("Sentence #")["Word"].apply(list).values
tags = df.groupby("Sentence #")["POS"].apply(list).values

# Step 4: Tokenization
tok = Tokenizer()
tok.fit_on_texts(sentences)
X = tok.texts_to_sequences(sentences)

tag_tok = Tokenizer()
tag_tok.fit_on_texts(tags)
y = tag_tok.texts_to_sequences(tags)

# Step 5: Padding
MAX_LEN = 128
X = pad_sequences(X, maxlen=MAX_LEN, padding='post')
y = pad_sequences(y, maxlen=MAX_LEN, padding='post')

# Step 6: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Step 7: Model
model = Sequential([
    Embedding(len(tok.word_index)+1, 64),
    LSTM(64, return_sequences=True),
```

```
    TimeDistributed(Dense(np.max(y)+1, activation='softmax'))
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])



# Step 8: Train
model.fit(X_train, y_train, epochs=5, batch_size=32)

# Step 9: Evaluate
loss, acc = model.evaluate(X_test, y_test)
print("Accuracy:", acc)

# Step 10: Prediction
sample = sentences[0]
seq = pad_sequences(tok.texts_to_sequences([sample]), maxlen=MAX_LEN, padding='post')

pred = np.argmax(model.predict(seq), axis=-1)[0]

idx2tag = {v:k for k,v in tag_tok.word_index.items()}

for w, t in zip(sample, pred[:len(sample)]):
    print(w, "->", idx2tag.get(t))
```